Clinical Rules Guide

This document explains how to write clinical decision rules using the s-expression language used in this codebase. Rules are written as Lisp-style s-expressions in `.lisp` files under `s_expression/`. The system compiles these into database queries that run against patient records.

Overview

There are three categories of rule file, each in its own subdirectory:

Directory	Purpose
`s_expression/tasks/`	Define clinical tasks: what a health worker should check for or do, given some trigger condition
`s_expression/system_diagnosis_rules/`	Define automated diagnostic reasoning: given patient evidence, what diagnosis should be assigned and with what certainty
`s_expression/system_priority_evaluations/`	Define triage priority: given a diagnosis or condition, what urgency level applies

Files are named by page number and topic (e.g. `20-anaphylaxis.lisp`). A single file may contain multiple rule expressions.

Comments use Lisp syntax: `;;`.

The Three Rule Types

1. Task

A task tells a health worker what to do (check for findings, take measurements, or manage with a treatment) once a trigger condition is met.

```
(task
  "Human-readable description"
  adult                          ;; age group
  <due_to>                       ;; trigger condition (s-expression)
  <action>                       ;; what to do
)
```

Age groups: `adult` | `all_ages` (children not yet implemented in apc-adult rules)

Actions:

`(check_for ...)` — list findings the health worker should ask about or observe:

```
(check_for
  (clinical_finding (snomed_concept "Stiff neck" "finding"))
  (clinical_finding (snomed_concept "Difficulty breathing" "finding"))
)
```

`(measure ...)` — a measurement to take:

```
(measure (measurement (snomed_concept "Hemoglobin saturation with oxygen"
"observable entity") %))
```

`(manage ...)` — a procedure or drug to administer, optionally gated by role:

```
(manage
  (snomed_concept "Oxygen therapy" "procedure")
)
(manage
  (snomed_concept "Product containing epinephrine" "medicinal product")
  (permission (role nurse))
)
```

Example — task triggered by a diagnosis:

```
(task
  "Check for urgent fever conditions"
  adult
  (active_condition (snomed_concept "Fever" "finding"))
  (check_for
    (clinical_finding (snomed_concept "Stiff neck" "finding"))
    (clinical_finding (snomed_concept "Drowsy" "finding"))
  )
)
```

Example — task triggered by a measurement:

```
(task
  "Check SpO₂ if respiratory rate >= 15 bpm"
  adult
  (>= (measurement (snomed_concept "Respiratory rate" "observable entity")
bpm) 15)
  (measure (measurement (snomed_concept "Hemoglobin saturation with
oxygen" "observable entity") %))
)
```

---

2. System Diagnosis Rule

A diagnosis rule specifies that when certain patient evidence is present, a particular diagnosis at a particular certainty level should be assigned.

```
(system_diagnosis_rule
  "Human-readable description"
  (diagnosis
    (snomed_concept "Disorder name" "disorder")
    <certainty>
  )
  adult
  <evidence>   ;; s-expression — the logical rule body
)
```

Certainty levels (most to least certain): `definite` | `probable` | `equivocal` | `possible` | `improbable`

Example — simple threshold:

```
(system_diagnosis_rule
  "Diagnose fever"
  (diagnosis (snomed_concept "Fever" "finding") definite)
  adult
  (>= (measurement (snomed_concept "Body temperature" "observable entity")
°C) 38)
)
```

Example — conjunction of findings and measurement:

```
(system_diagnosis_rule
  "Diagnose probable hypertensive emergency"
  (diagnosis (snomed_concept "Hypertensive emergency" "disorder")
probable)
  adult
  (and
    (or
      (>= (measurement (snomed_concept "Systolic blood pressure"
"observable entity") mmHg) 180)
      (>= (measurement (snomed_concept "Diastolic blood pressure"
"observable entity") mmHg) 110)
    )
    (or
      (clinical_finding (snomed_concept "Headache" "finding"))
      (clinical_finding (snomed_concept "Difficulty breathing" "finding"))
    )
```

```
    )
)
```

Example — `any2` (at least 2 of N conditions):

```
(system_diagnosis_rule
  "Diagnose possible anaphylaxis"
  (diagnosis (snomed_concept "Anaphylaxis" "disorder") possible)
  adult
  (any2
    (clinical_finding (snomed_concept "Insect sting" "disorder"))
    (clinical_finding (snomed_concept "Itching" "finding"))
    (clinical_finding (snomed_concept "Difficulty breathing" "finding"))
    (clinical_finding (snomed_concept "Collapse" "finding"))
  )
)
```

Multiple `system_diagnosis_rule` blocks in the same file are independent rules that each run separately.

---

3. System Priority Evaluation

Assigns a triage priority level when condition(s) are present.

```
(system_priority_evaluation
  "Human-readable description"
  all_ages
  <priority>
  <condition>
)
```

Priority levels: `Emergency` | `Very urgent` | `Urgent`

Example:

```
(system_priority_evaluation
  "Urgent: probable anaphylaxis"
  all_ages
  Urgent
  (diagnosis (snomed_concept "Anaphylaxis" "disorder") probable)
)
```

S-Expression Language Reference

`snomed_concept`

All clinical concepts reference SNOMED CT by name and category:

```
(snomed_concept "Anaphylaxis" "disorder")
(snomed_concept "Systolic blood pressure" "observable entity")
(snomed_concept "Face structure" "body structure")
(snomed_concept "Sudden onset" "qualifier value")
```

Common categories: `"finding"` | `"disorder"` | `"observable entity"` | `"body structure"` | `"qualifier value"` | `"morphologic abnormality"` | `"substance"` | `"procedure"` | `"medicinal product"` | `"clinical drug"`

SNOMED concept names are matched against the database. When part of the <due_to> concepts will also matches *descendants* in the SNOMED hierarchy — so querying `"Nasal discharge"` will also match more specific findings like `"Purulent nasal discharge"`.

`clinical_finding`

The most common way to assert that a patient has a clinical observation from this encounter. It is syntactic sugar for a `finding` with `root_snomed_concept = "Clinical finding"`.

```
(clinical_finding (snomed_concept "Abdominal pain" "finding"))
```

With a qualifier:

```
(clinical_finding
  (snomed_concept "Swelling" "finding")
  (qualifier (snomed_concept "Sudden onset" "qualifier value"))
)
```

With a `finding_site` attribute:

```
(clinical_finding
  (snomed_concept "Swelling" "finding")
  (finding_site (snomed_concept "Face structure" "body structure"))
  (qualifier (snomed_concept "Sudden onset" "qualifier value"))
)
```

Modifiers can appear in any order. Multiple qualifiers and attributes are all required (AND semantics).

`finding`

The general form, used when you need to express relationships that aren't standard clinical findings. Accepts a root concept, a specific concept, and optional modifiers.

```
(finding
  (snomed_concept "Exposure to (contextual qualifier)" "qualifier value")
  (snomed_concept "Fish" "substance")
)
```

The above matches a patient record where the root concept is "Exposure to…" and the specific concept is "Fish".

`allergy`

Shorthand for an allergic condition with a causative agent. Compiles to a `finding` with:

- root: `Clinical finding`
- specific: `Allergic condition`
- attribute `Causative agent = <substance>`
- `history: true` (searches the patient's full history, not just the current encounter)

```
(allergy (snomed_concept "Fish" "substance"))
(allergy (snomed_concept "Peanut" "substance"))
```

`measurement` and comparison operators

Used to match numeric measurements recorded against a patient.

```
(measurement (snomed_concept "Systolic blood pressure" "observable
entity") mmHg)
```

Available units: `%` | `bpm` | `°C` | `cm` | `kg` | `mmol/L` | `mmHg` | `mm`

Measurements are used inside comparison expressions:

```
(>= (measurement (snomed_concept "Body temperature" "observable entity")
°C) 38)
(<  (measurement (snomed_concept "Systolic blood pressure" "observable
entity") mmHg) 90)
(=  (measurement (snomed_concept "Respiratory rate" "observable entity")
bpm) 15)
```

Operators: `>` | `<` | `>=` | `<=` | `=`

`diagnosis`

Asserts that a patient has been diagnosed with a condition at a given certainty level. In rule bodies (the `<evidence>` or `due_to` position), this compiles to a query against patient evaluation records.

```
(diagnosis (snomed_concept "Anaphylaxis" "disorder") probable)
(diagnosis (snomed_concept "Fever" "finding") definite)
```

Certainty levels: `definite` | `probable` | `equivocal` | `possible` | `improbable`

`active_condition`

Checks whether a condition is currently active for the patient — either as a historical/recorded finding, or as a diagnosis at probable or definite certainty, or as a "Yes" for self-reported status. This is the right expression to use in a task's `due_to` when you want it to fire for patients who have an ongoing condition that has not resolved either from this encounter or past encounters.

```
(active_condition (snomed_concept "Anaphylaxis" "disorder"))
(active_condition (snomed_concept "Fever" "finding"))
```

The optional `possible` modifier widens the check to include possible and equivocal diagnoses:

```
(active_condition (snomed_concept "Anaphylaxis" "disorder") possible)
```

Compiled expansion — `(active_condition X)` expands to:

```
(or
  (history (clinical_finding X))
  (history (finding <self-reported-status-attribute> X Yes))
  (diagnosis X probable)
  (diagnosis X definite)
)
```

With `possible`, also includes `(diagnosis X equivocal)` and `(diagnosis X possible)`.

`history`

Wraps a `clinical_finding` or `finding` to search the patient's full history (all encounters), not just the current one. Generally you'd prefer `active_condition` which is more general and uses `history` under the hood.

```
(history (clinical_finding (snomed_concept "Asthma" "disorder")))
```

Logical operators

`or` — at least one child must be satisfied:

```
(or
  (clinical_finding (snomed_concept "Headache" "finding"))
  (clinical_finding (snomed_concept "Dizziness" "finding"))
)
```

`and` — all children must be satisfied:

```
(and
  (clinical_finding (snomed_concept "Stiff neck" "finding"))
  (active_condition (snomed_concept "Fever" "finding"))
)
```

`any2` — at least 2 of the children must be satisfied:

```
(any2
  (clinical_finding (snomed_concept "Insect bite - wound" "disorder"))
  (clinical_finding (snomed_concept "Itching" "finding"))
  (clinical_finding (snomed_concept "Difficulty breathing" "finding"))
  (clinical_finding (snomed_concept "Collapse" "finding"))
)
```

`not` — the child must not be satisfied. Use sparingly; prefer `excluding` on a `finding` for simple exclusion cases.

`qualifier`

Modifies a `clinical_finding` or `finding` by requiring an associated qualifier record (e.g. onset, severity, laterality).

```
(qualifier (snomed_concept "Sudden onset" "qualifier value"))
(qualifier (snomed_concept "Severe (severity modifier)" "qualifier
value"))
```

Multiple qualifiers on a finding all need to be present.

`finding_site`

Shorthand attribute specifying anatomical location. Used inside `clinical_finding`:

```
(clinical_finding
  (snomed_concept "Swelling" "finding")
  (finding_site (snomed_concept "Tongue structure" "body structure"))
)
```

This compiles to an `attribute` check. It can be satisfied either by an explicit attribute record on the patient finding, or by SNOMED relationship inference (if the concept's definition already implies the site).

`excluding`

Excludes records that match another expression. Applied directly within a `finding`:

```
(finding
  (snomed_concept "Clinical finding" "finding")
  (snomed_concept "Burn" "disorder")
  (excluding (finding (snomed_concept "Clinical finding" "finding")
(snomed_concept "Inhalation burn" "disorder")))
)
```

How the Compiler Translates Expressions to Queries

Understanding how each expression type maps to the database helps when debugging or extending rules.

All queries run against `patient_records_aggregated` — a denormalized table with one row per patient record, indexed for fast lookup. Three of its "subsets" are joined depending on the expression type (see the next section).

`clinical_finding` / `finding` → `patient_findings`

Compiles to a SELECT from `patient_records_aggregated` INNER JOINed to `patient_findings`, filtered by root/specific SNOMED concept IDs, existence, qualifiers, and attributes.

Non-exact specific concept matches use `snomed_concept_active_descendants_realized` to include descendants.

`diagnosis` → `patient_evaluations`

`diagnosis` is first converted via `diagnosisToEvaluation` to an `evaluation` node: it queries `patient_evaluations` for an evaluation record whose specific concept matches the diagnosed disorder and whose value concept is the appropriate certainty qualifier (e.g. "probable").

`evaluation` → `patient_evaluations`

Directly queries `patient_evaluations`. If the evaluation has an `evaluates` clause, it further requires that the evaluation's `evaluates_record_id` points to a record satisfying the inner expression.

`measurement` → `patient_findings` + `patient_measurements`

Queries records joined to `patient_measurements`, filtered by SNOMED concept and units. Measurement comparisons (e.g. `>=`) apply numeric WHERE clauses on `patient_measurements.value`, accounting for the stored comparator (e.g. a stored `>=` value of 90 satisfies a query for `>= 90`).

`procedure` → `patient_procedures`

Queries records joined to `patient_procedures`, filtered by specific SNOMED concept.

`active_condition` → OR expansion

Expanded at query time to an OR of history finding checks plus diagnosis checks at probable/definite certainty (and optionally equivocal/possible if `possible` is set).

`allergy` → `finding` with attributes

Expanded to a `finding` with root=`Clinical finding`, specific=`Allergic condition`, and a `Causative agent` attribute. The `history: true` flag means it searches all encounters.

`patient_records_aggregated` and Its Three Subsets

`patient_records_aggregated` is a denormalized summary table with one row per patient record. It is populated automatically by a database trigger on INSERT to `patient_records`. Every patient clinical record — regardless of type — lands here.

Schema highlights:

Column	Description
`id`	UUID, same as `patient_records.id`
`patient_id`	Patient
`patient_encounter_id`	Encounter during which the record was created
`root_snomed_concept_id/name/category`	Top-level clinical concept (e.g. "Clinical finding")
`specific_snomed_concept_id/name/category`	The specific concept (e.g. "Anaphylaxis")
`existence`	`Yes` | `No` | `Unknown`
`value`	JSONB — type-specific payload (see below)

The `existence` column is derived from the `value_snomed_concept_id` on the underlying `patient_records` row: the special SNOMED concepts `NO_QUALIFIER` and `UNKNOWN_QUALIFIER` yield `No` and `Unknown`; everything else yields `Yes`.

The `value` JSONB field holds one of: `snomed_concept`, `event`, `measurement`, `score`, `link`, or `task`.

The Three Subsets

The table itself is undifferentiated, but queries distinguish record type by joining to one of three sub-tables:

Findings (`patient_findings`)

Clinical observations: what the patient has or does not have. A finding record may belong to a specific procedure encounter (`procedure_id`). Findings also have a sub-type for measurements (joined via `patient_measurements`) which stores a numeric `value`, `units`, and `comparator`.

```
-- conceptually:
patient_records_aggregated
  INNER JOIN patient_findings ON patient_findings.id =
patient_records_aggregated.id
```

Evaluations (`patient_evaluations`)

Assessment outcomes: diagnoses, scores, and other derived conclusions. An evaluation may reference the record it evaluates via `evaluates_record_id`, and may further have scores in `patient_evaluation_scores`.

```
patient_records_aggregated
  INNER JOIN patient_evaluations ON patient_evaluations.id =
patient_records_aggregated.id
```

A `diagnosis` expression is always compiled as an evaluation query — the certainty level (probable, definite, etc.) is stored as the `value_snomed_concept` on the evaluation record.

Procedures (`patient_procedures`)

Clinical actions that were performed: check_for sessions, administered treatments, ordered measurements. A procedure may have an associated link (`patient_record_links`) or an embedded s-expression (`patient_record_s_expressions`) describing what was done.

```
patient_records_aggregated
  INNER JOIN patient_procedures ON patient_procedures.id =
patient_records_aggregated.id
```

Supporting Tables

Table	Role

Table	Role
`patient_record_qualifiers`	Qualifier/attribute records that modify a main record (e.g. "Sudden onset" qualifying "Swelling")
`patient_record_relations`	Generic source→destination relationships between records
`patient_records_still_valid`	Table with the ids of patient records that haven't been marked as entered in error
`snomed_concept_active_descendants_realized`	Pre-computed transitive closure of SNOMED hierarchy, used for ancestor matching

Known SNOMED Concepts and Why They Matter for Rules

How `due_to` makes rules fire

Tasks, diagnosis rules, and priority evaluations only execute when their `due_to` (or `condition`) expression is satisfied by the patient's current records. If the s-expression in `due_to` refers to a concept that is never recorded, the rule never fires — no matter how well the rule body is written.

The most reliable way to ensure a rule fires is to anchor its `due_to` to a concept that the system is already recording for the right patients. Three places in the codebase define the canonical s-expressions for the concepts the system tracks:

`shared/warning_signs.ts` — WARNING_SIGN_DEFS

Warning signs are the first things recorded for a patient at triage. Each entry has a `clinical_finding_s_expression` — the exact s-expression used to record that finding. Rules that should fire immediately for high-acuity patients must reference these concepts in their `due_to`.

Selected entries:

Key	`clinical_finding_s_expression`	Priority
`Obstructed airway`	`(clinical_finding (snomed_concept "Respiratory obstruction" "disorder"))`	Emergency
`Cardiac arrest`	`(clinical_finding (snomed_concept "Cardiac arrest" "disorder"))`	Emergency
`Seizure`	`(clinical_finding (snomed_concept "Seizure" "finding"))`	Emergency
`Burn Facial`	`(clinical_finding (snomed_concept "Burn of face" "disorder"))`	Emergency
`Burn Inhalation`	`(clinical_finding (snomed_concept "Inhalation burn due to hot gas" "disorder"))`	Emergency

Key	clinical_finding_s_expression	Priority
Acute shortness of breath	(clinical_finding (snomed_concept "Dyspnea" "finding") (qualifier (snomed_concept "Severe (severity modifier)" "qualifier value")))	Very urgent
Chest pain	(clinical_finding (snomed_concept "Chest pain" "finding"))	Very urgent
Focal neurology	(clinical_finding (snomed_concept "Cerebrovascular accident" "disorder"))	Very urgent
Poisoning	(clinical_finding (snomed_concept "Poisoning" "disorder"))	Very urgent
Haemorrhage Uncontrolled	(clinical_finding (snomed_concept "Bleeding" "finding") (qualifier (snomed_concept "Uncontrolled" "qualifier value")))	Very urgent
Severe pain	(clinical_finding (snomed_concept "Severe pain" "finding"))	Very urgent
Vomiting fresh blood	(clinical_finding (snomed_concept "Vomiting blood - fresh" "disorder"))	Very urgent
Coughing blood	(clinical_finding (snomed_concept "Hemoptysis" "finding"))	Very urgent
Burn Circumferential	(clinical_finding (snomed_concept "Burn" "disorder") (qualifier (snomed_concept "Circumferential" "qualifier value")))	Very urgent

For a task that should fire when a patient arrives with chest pain, the `due_to` should be:

```
(clinical_finding (snomed_concept "Chest pain" "finding"))
```

not a paraphrase. The exact concept name must match.

`shared/common_symptoms.ts` — COMMON_SYMPTOMS

Common symptoms are the self-reported presenting complaints collected via the chatbot. These are what most APC patients arrive with. Rules for routine conditions (non-emergency) should anchor their `due_to` to these symptoms.

Key	clinical_finding_s_expression
Fever	(clinical_finding (snomed_concept "Fever" "finding"))
Cough	(clinical_finding (snomed_concept "Cough" "finding"))

Key	clinical_finding_s_expression
Nasal discharge	(clinical_finding (snomed_concept "Nasal discharge" "finding"))
Sore throat	(clinical_finding (snomed_concept "Sore throat" "finding"))
Headache	(clinical_finding (snomed_concept "Headache" "finding"))
Fatigue	(clinical_finding (snomed_concept "Fatigue" "finding"))
Dyspnea	(clinical_finding (snomed_concept "Dyspnea" "finding"))
Nausea	(clinical_finding (snomed_concept "Nausea" "finding"))
Vomiting	(clinical_finding (snomed_concept "Finding of vomiting" "finding"))
Diarrhea	(clinical_finding (snomed_concept "Diarrhea" "finding"))
Dizziness	(clinical_finding (snomed_concept "Dizziness" "finding"))
Muscle pain	(clinical_finding (snomed_concept "Muscle pain" "finding"))
Insect bite	(clinical_finding (snomed_concept "Insect bite - wound" "disorder"))
Backache	(clinical_finding (snomed_concept "Backache" "finding"))
Constipation	(clinical_finding (snomed_concept "Constipation" "finding"))

The `Fever` task file (`s_expression/tasks/apc-adult/24-fever.lisp`) is a good example: it uses `(active_condition (snomed_concept "Fever" "finding"))` in `due_to`, matching exactly the concept recorded by the common symptom.

`shared/snomed_concepts.ts` — source

This file defines named TypeScript constants for all SNOMED concepts that have special meaning somewhere in the rest of the application.

Vital sign observables (use in `measurement`):

Constant	Name	Category
BODY_TEMPERATURE	"Body temperature"	"observable entity"
SYSTOLIC_BLOOD_PRESSURE	"Systolic blood pressure"	"observable entity"
DIASTOLIC_BLOOD_PRESSURE	"Diastolic blood pressure"	"observable entity"
HEMOGLOBIN_SATURATION_WITH_OXYGEN	"Hemoglobin saturation with oxygen"	"observable entity"

Constant	Name	Category
`RESPIRATORY_RATE`	`"Respiratory rate"`	`"observable entity"`
`PULSE_FUNCTION`	`"Pulse, function"`	`"observable entity"`
`BLOOD_GLUCOSE_STATUS`	`"Blood glucose status"`	`"observable entity"`

Common chronic conditions (use with `active_condition` or `diagnosis`):

Constant	Name	Category
`DIABETES_MELLITUS`	`"Diabetes mellitus"`	`"disorder"`
`TUBERCULOSIS`	`"Tuberculosis"`	`"disorder"`
`HUMAN_IMMUNODEFICIENCY_VIRUS_INFECTION`	`"Human immunodeficiency virus infection"`	`"disorder"`
`ASTHMA`	`"Asthma"`	`"disorder"`
`CHRONIC_OBSTRUCTIVE_PULMONARY_DISEASE`	`"Chronic obstructive pulmonary disease"`	`"disorder"`
`HEART_DISEASE`	`"Heart disease"`	`"disorder"`
`EPILEPSY`	`"Epilepsy"`	`"disorder"`
`PREGNANCY`	`"Pregnancy"`	`"finding"`

Qualifier values (use with `qualifier`):

Constant	Name
`SUDDEN_ONSET`	`"Sudden onset"`
`CHRONIC`	`"Chronic"`

Tips for Writing Rules

- **Use `active_condition` in task `due_to` for ongoing conditions** — it checks history and diagnosis. Use raw `clinical_finding` only when you specifically mean a finding in the current encounter.
- **Prefer `clinical_finding` over `finding`** — `clinical_finding` is clearer and covers the vast majority of cases.
- **Use `allergy` for substance allergies** — don't try to hand-write the equivalent `finding`; `allergy` is the canonical form and will keep the compiled query correct.
- **Anchor `due_to` to recorded concepts** — check `WARNING_SIGN_DEFS`, `COMMON_SYMPTOMS`, and `shared/snomed_concepts.ts` first. If your trigger concept isn't in one of those, the rule may never fire in practice.

- **Add `;;` comments** to explain clinical rationale, especially for non-obvious thresholds or `any2` counts.
- **File naming convention** — match the page number from the clinical protocol and use a descriptive slug: `132-hypertension.lisp`.
- **Multiple rules per file are fine** — tasks and diagnosis rules that belong to the same clinical topic can coexist in one file.
- **Test SNOMED names against the database** — concept names are matched exactly (case-sensitive). Run `deno task test ./test/shared/compiled_s_expressions.test.ts` to validate all concepts in `.lisp` files exist in the database.